DNSSEC_Signature_5_working

You are an DNS security expert. Here is an example DNS attack that effects the cache of the resolver to reduce its impact.
Example:
NS cache flush attack
Summary:
Attack owns an auhtoritative server and control example.com. Domain.
The attacker issues A type queries for different unique names under this domain. For example, test1.example.com, test2.example.com test10000.example.com
The attacker's authoritative server responds this queries with around 1500 NS records thus making the DNS reply 65k(the maximum value). Each of these values are cached by the resolver, thus evicting the benign cached records from the resolver's cache causing resolver to requery these domains thus reducing its performance significantly.

I want you to come with another cache flushing attack based on DNSSEC.
utilize your DNSSEC knowledge and check anything can be used against to a resolver for cache flushing purposes.

Below is a hypothetical DNSSEC-based cache-flushing attack. The high-level idea is similar to the NS cache-flush attack (i.e., generating voluminous records to fill up the resolver's cache), but leverages DNSSEC's larger record sizes and the overhead of cryptographic validation.

Attack Overview

1. **Attacker's Prerequisite**
    - Owns a malicious DNSSEC-signed zone (e.g., `example-sec.com`).
    - Has control over the zone's authoritative name servers.

- Configures the zone with deliberately large DNSSEC records (e.g., multiple large RSA/ECDSA keys, multiple signatures, inflated NSEC/NSEC3 chains).

2. **Goal**
   - Flood the resolver's cache with large (and mostly useless) DNSSEC records and negative (NXDOMAIN) responses, thereby evicting or overshadowing legitimate cached entries.
   - Force the resolver to re-query or re-validate benign domains, reducing overall resolver performance and increasing upstream load.

Detailed Attack Mechanics

1. **Overly Large DNSKEY Set**
   - The attacker's domain (`example-sec.com`) is signed with multiple large DNSKEY records. For instance, it can include:
     - Multiple RSA-4096 keys, or
     - Several elliptic-curve keys with large key sizes, or
     - A mix of RSA/ECDSA/EdDSA algorithms for maximum complexity.
   - Each DNSKEY RRset might be tens of kilobytes when combined (especially if multiple RRSIGs are present).
2. **Excessive or Complex DS Records**
   - The zone can also be configured to have multiple DS records at the parent zone. While the parent typically controls DS records, an attacker who controls both the child zone and a cooperating parent (in some contrived scenario) could set up multiple DS records for each key, further increasing the size of validation chains.
3. **Crafted NSEC/NSEC3 Chains**
   - For non-existing subdomains, DNSSEC requires returning proof of non-existence via NSEC or NSEC3 records. These can be manipulated to be large or fragmented:
     - **NSEC-based approach**: The attacker splits up the zone into many discrete 'gaps' so that returning proof of non-existence references large chunks of the zone (i.e., multiple NSEC records).

- **NSEC3-based approach**: Use high iteration counts, salt, and multiple NSEC3 parameters, increasing the size and validation effort for negative responses.

4. **Queries for Random Non-Existent Names**
   - Similar to the classic “cache flush” pattern, the attacker (or a botnet under their control) issues A/AAAA queries for random and unique subdomains under `example-sec.com`:

     ```python
     test1.example-sec.com
     test2.example-sec.com
     ...
     test10000.example-sec.com
     ```

   - Since these subdomains do not exist, each query results in:
     - A DNSSEC-signed NXDOMAIN response.
     - Large NSEC/NSEC3 proofs for non-existence.
     - The full DNSKEY RRset (if the resolver does not already have it cached or if the TTL has expired).
5. **Cache Pollution and Eviction**
   - Each negative answer (NXDOMAIN) is cached along with its signature and proof of non-existence (NSEC/NSEC3).
   - The bulky DNSKEY RRsets and RRSIGs also occupy significant space.
   - Repeated queries for different names quickly fill the resolver’s cache with large negative entries, displacing “benign” DNS records (A/AAAA records for popular domains, MX records, etc.).
   - This triggers the resolver to re-fetch or re-validate legitimate domain data more frequently, increasing its workload and latency.
6. **Validation Overhead**
   - On top of consuming cache memory, the resolver must cryptographically validate these large DNS responses.
   - Repeated validation requests add CPU overhead on the resolver side, further degrading performance.

Why This Works

- **DNSSEC Records Are Large**
  Compared to plain DNS, DNSSEC adds DNSKEY, RRSIG, NSEC/NSEC3, and DS records. These can make individual DNS responses very large (often >1 KB, sometimes approaching the 64 KB UDP limit).
- **Negative Caching**
  Even negative answers (NXDOMAIN) get cached. Attackers exploit this by generating thousands of unique subdomains, causing many negative entries plus their associated DNSSEC proofs to occupy the cache.
- **Resolver Resource Constraints**
  Resolvers typically have a finite cache size and eviction policies (e.g., LRU or TTL-based). Large NXDOMAIN entries and DNSSEC data take space in the cache. Legitimate cached entries get flushed out more quickly, leading to extra lookups and higher latency for real queries.
- **Validation Cost**
  The resolver must perform cryptographic validation of these records, adding CPU overhead. With sufficiently large or complex keys, the validation overhead can be non-trivial.

Mitigations

1. **Limit Negative Caching Size**
   - Configure the resolver to cap the number of negative entries in its cache. Some resolvers allow separate memory pools for negative vs. positive cache entries.
2. **DNS Response Size and QNAME Minimization**
   - Encourage use of response size limits, EDNS(0) padding policies, or enforcing minimal response sizes for negative answers.
   - QNAME minimization can limit the amount of zone data requested upstream.
3. **Rate Limiting and Access Controls**

- Implement query-rate limits or detection of suspicious patterns (many random subdomains under the same zone).
- Blackhole known abusive or suspicious domains.

4. **Resolvers with Intelligent Eviction**
    - Resolvers could implement more sophisticated eviction logic, e.g. lower priority for caching negative NXDOMAIN + DNSSEC combos, or shorter TTL enforcement for suspicious large negative responses.
5. **Operational Best Practices**
    - Monitor DNSSEC zone sizes and design (avoid extremely large key sets, keep negative proofs small).
    - Encourage authoritative servers to avoid unnecessarily large DNSKEY sets.
    - DNSSEC-signed zones should only publish essential keys and minimal NSEC ranges.

Summary

By exploiting the inherently larger record sizes of DNSSEC (DNSKEY, RRSIG, NSEC/NSEC3), an attacker can craft responses that quickly balloon a resolver's cache. Repeated queries for random, non-existent subdomains under a DNSSEC-signed domain cause massive NXDOMAIN responses to be cached, displacing genuine entries and imposing high validation overhead. The net effect is a denial-of-service-like impact on the resolver, akin to "cache flushing," but made more potent by DNSSEC's cryptographic baggage and bulky record sets.

The attacker's domain (example-sec.com) is signed with multiple large DNSKEY records. For instance, it can include:
Multiple RSA-4096 keys, or
Several elliptic-curve keys with large key sizes, or
A mix of RSA/ECDSA/EdDSA algorithms for maximum complexity.
Each DNSKEY RRset might be tens of kilobytes when combined (especially if multiple RRSIGs are present).

Focus here, how many keys the attacker need to achieve 65KB response (single response)?

Below is a practical way to ballpark how many large DNSKEY records you'd need to push a single DNS response up to (or above) 65 KB.

---

1. Estimating the Size of One RSA-4096 DNSKEY Record

The exact size of an individual DNSKEY record depends on:

- **Key bit length** (e.g., RSA-4096 has a 4096-bit modulus, which is 512 bytes just for the modulus).
- **Public exponent** size (often small, e.g., 65537).
- **Base64 encoding overhead** (Base64 expands raw bytes by roughly 4/3).
- **DNS wire-format overhead** (owner name, type/class/flags/TTL, key flags, protocol, algorithm fields, etc.).

A commonly cited rule of thumb for RSA-4096 DNSKEY is around **550–700 bytes** once you factor in base64 plus the DNS overhead. In many real-world measurements, you'll see each RSA-4096 DNSKEY land somewhere in this range.

---

2. Don't Forget RRSIG Overhead

For the DNSKEY RRset, there will be one or more **RRSIG(DNSKEY)** records. In practice:

- **One RRSIG per algorithm** used to sign the DNSKEY RRset.
- Each RRSIG is also large because it includes an RSA-4096 signature (another 512 raw bytes, plus overhead, base64-encoded).

Hence, if you have N DNSKEY records all using the same algorithm, you'll still have **one** RRSIG covering the entire DNSKEY RRset (assuming it's the same key signing the set). If you mix algorithms (e.g., RSA + ECDSA + EdDSA), you add additional RRSIGs.

A single RSA-4096 RRSIG can also be in the **550–700 bytes** range.

---

3. Approximate Calculation

Let's assume each DNSKEY is ~600 bytes on the wire and the single RRSIG adds another ~600 bytes. Then:

- **Size for each DNSKEY:** 600 bytes
- **Size for a single RRSIG:** 600 bytes total, but (important!) you typically only have one RRSIG *per algorithm* covering the set.

Case A: Only One Algorithm (All RSA-4096)

- If you have N DNSKEY records, total size $\approx 600 \times N + 600$ (for the one RRSIG).

  We want this $\geq 65{,}000$ bytes:

$$600 \times N + 600 \geq 65{,}000$$

$$600 \times N \geq 64{,}400$$

$$N \geq \frac{64{,}400}{600} \approx 107$$

  So, roughly **107 DNSKEY records** (plus the single RRSIG) could nudge you just beyond 65 KB.

Case B: Multiple Algorithms (e.g., RSA-4096 + ECDSA)

If you intentionally mix multiple algorithms (to add complexity or inflate size), you'll have:

- N DNSKEY records total, split among different algorithms.
- 1 RRSIG per algorithm. If you have 2 algorithms, you have 2 RRSIGs, etc.

This can slightly reduce the number of DNSKEYs needed, because each additional RRSIG is another large record. For example, with 2 algorithms and 2 RRSIGs, you add ~1200 bytes just in signatures.

If each DNSKEY is still ~600 bytes, you might hit 65 KB with fewer total keys because of the overhead from multiple RRSIG records.

4. Realistic Range

Depending on your exact key sizes, base64 overhead, and the number of algorithms:

- **Single-algorithm scenario**: ~100+ large DNSKEY records can blow up a single response to 65 KB or more.
- **Multi-algorithm scenario**: Fewer total DNSKEY records may be needed (on the order of 80–90) because each extra algorithm adds one more large RRSIG.

In short, to reliably exceed 65 KB **with RSA-4096 alone**, plan on having on the order of **100 DNSKEY RRs** in one RRset. If you add multiple algorithms (each with its own RRSIG), you could push the total size over 65 KB with *slightly* fewer keys—perhaps in the **80–90** range.

Bottom Line

- **One algorithm (RSA-4096 only)**: $100+$ DNSKEYs is a good rule of thumb to exceed 65 KB in a single DNSKEY RRset response.
- **Mixed algorithms**: Fewer DNSKEYs might suffice (e.g., **80–90**), since each added algorithm also contributes an extra (large) RRSIG record.

How many RSA-4096 keys can fit into a 65k DNS response, including all the RRSIGs?

3/3

Below is a rough way to estimate how many RSA-4096 DNSKEY records (plus the necessary RRSIG) you can cram into a single 65 KB DNS response. The short answer: **on the order of 90–110 keys** if you only use one algorithm (RSA-4096) and have a single RRSIG covering them all.

---

1. Why “90–110” Instead of One Exact Number?

Exact sizes vary because of:

- **DNS overhead** (headers, question section, name compression differences, etc.).
- **Base64 encoding** (which adds about 33% overhead).
- **Key details** (the modulus size is 512 bytes, exponent size can vary, etc.).
- **RRSIG overhead** (for RSA-4096, the signature alone is another 512 bytes raw).

Different resolvers and authoritative servers might format responses slightly differently, so the size per DNSKEY can bounce between ~550 bytes and ~700 bytes in practical scenarios.

---

2. Example Calculation

A commonly cited rule of thumb:

1. **Single RSA-4096 DNSKEY**
   - About **600 bytes** total on the wire (including flags, algorithm, base64 overhead, etc.).
2. **One RRSIG(DNSKEY) record**
   - Also around **600 bytes**, because it holds a 4096-bit signature (512 raw bytes + base64 overhead + some DNS fields).
3. **DNS message overhead**
   - Header (12 bytes), question section, and any extra sections. Realistically 100–200 bytes in total.

If we assume:

- 600 bytes per DNSKEY
- 600 bytes for the single RRSIG
- ~100 bytes of miscellaneous DNS overhead

Then the total size S for N DNSKEYs is approximately:

$$S \approx (\text{DNSKEY size} \times N) + \text{RRSIG} + \text{overhead}$$

$$S \approx (600 \times N) + 600 + 100$$

We want $S \leq 65{,}535$ bytes (the practical UDP limit without fragmentation). Setting $S = 65{,}000$ for simplicity:

$$600 \times N + 600 + 100 \approx 65{,}000$$

$$600 \times N \approx 64{,}300$$

$$N \approx 107$$

So **107** DNSKEY records (plus one RRSIG) can push the response right to ~65 KB. Depending on exact overhead, you might end up with anywhere from **90** to **110** DNSKEYs in practice.

---

3. What If You Add Multiple Algorithms?

If you add a second algorithm (e.g., ECDSA) just to inflate the DNSKEY RRset size, you'll have **multiple RRSIG** records—one RRSIG per algorithm. That extra signature data can push you to 65 KB with fewer overall DNSKEYs. In other words, mixing algorithms can "cheat" the count downward because each new algorithm adds another (potentially large) RRSIG record.

Bottom Line

- **Single Algorithm (RSA-4096)**: Around **100 DNSKEYs** (give or take) will hit the 65 KB limit.
- **Multiple Algorithms**: Fewer DNSKEYs are needed because each added algorithm brings another large RRSIG record.

This is enough to bloat a single DNS response to its maximum size and set the stage for the kind of cache-flushing or resource-exhaustion attack discussed earlier.

2/2

How about A type RRsets, how many RRSIGs we will have in this case?

2/2

When talking specifically about **A (address) RRsets** under DNSSEC, the **number of RRSIG records** that appear depends on how many active signing keys (and algorithms) are used to sign that RRset. Here's the breakdown:

---

1. Typical Single-Algorithm, Single-ZSK Scenario

- **One ZSK (Zone Signing Key).**
  - In most standard DNSSEC deployments, there is **one** active ZSK that signs all resource records (A, AAAA, MX, etc.) in the zone.
  - As a result, **each RRset** (including the A RRset) has **exactly one RRSIG** generated by that ZSK.
- **KSK (Key Signing Key)** does **not** directly sign the A RRset.
  - The KSK typically only signs the DNSKEY RRset (i.e., the set of keys).
  - Therefore, it does **not** create an extra signature on normal data like A records.

Hence, in the **common single-ZSK, single-algorithm setup**, an A RRset will have **1 RRSIG**.

---

2. Multiple Algorithms or Multiple ZSKs

If the zone is signed with more than one algorithm (e.g., RSA-2048 *and* ECDSA P-256) or if you have multiple active ZSKs for the same algorithm, you could see **several RRSIGs** on the same A RRset:

1. **Multi-Algorithm:**
   - Each algorithm used to sign the zone generates its own signature.
   - If you have 2 algorithms, expect **2 RRSIGs** per RRset, and so on.
2. **Multiple Active ZSKs (key rollover or special setups):**
   - If you temporarily have 2 or more active ZSKs for the same algorithm (e.g., during a rollover where old key and new key are both signing), each key generates its own RRSIG.

- This could lead to **2 (or more) RRSIGs** for the same RRset, even under a single algorithm.

---

3. Practical Answer

- **Most zones** use exactly **1 ZSK** (one algorithm), resulting in **1 RRSIG per A RRset**.
- With **multiple ZSKs** or **multiple algorithms**, you might see **2 or more RRSIGs** on the A RRset, one for each active key/algorithm.

In short, **there is no fixed limit** in the DNSSEC specification on how many RRSIGs can exist for a single RRset. The actual count depends on the zone's key management and the algorithms in use. However, the most common real-world case is **just 1** signature for A RRsets.

In the case I have 100 ZKS, how many RRSIGs?

If you truly have **100 active ZSKs** (all signing the same RRset), you would end up with **100 RRSIG records** on that RRset—one signature per active key.

In DNSSEC, every **active** key that is designated a Zone Signing Key (ZSK) signs all RRsets, producing one RRSIG per RRset per key. So if you have 100 ZSKs, each one signs the A RRset, leading to 100 corresponding RRSIGs.

So would 100 RSA 4096 ZSK be over max DNS Packet size 65KB

2/2

It's very likely that **100 RSA-4096 ZSKs**, each generating its own RRSIG for an A RRset, **would exceed** the 65 KB DNS message size limit (or come extremely close). Here's why:

1. **Approximate Size per RSA-4096 RRSIG**
   - An RSA-4096 signature alone (the "signature" part) is 512 bytes in raw form.
   - Once you add base64 encoding and DNS overhead (algorithm/expiration fields, etc.), **each RRSIG** commonly ends up around **550–700 bytes** on the wire. A middle-ground

estimate is about **600 bytes**.

2. **Multiply by 100 Signatures**
    - If you have **100 ZSKs**, you get **100 RRSIG** records on the A RRset.
    - Using the rough 600-byte estimate:
$$100 \times 600 \text{ bytes} = 60{,}000 \text{ bytes}$$
3. **Add the A Record + DNS Overhead**
    - You still need space for the question section, the actual A RR, name compression pointers, EDNS(0) options, etc. Even if that's "only" a few hundred bytes in total, it pushes you well over 60 KB.
4. **Conclusion: Very Close to or Above 65 KB**
    - At ~60 KB just in signatures, plus the rest of the DNS packet overhead, you'd almost certainly breach the **65,535-byte** theoretical maximum for a single UDP DNS response.
    - In reality, large responses get truncated and force TCP usage, or they get fragmented at the IP layer (which brings its own reliability issues).

Hence, **yes**, 100 RSA-4096 ZSKs signing the same A RRset can easily push a DNS response beyond the 65 KB limit.